

前言

LSM-tree和B-tree作为两种不同的数据存储索引结构，在不同的情况下有着各自的优势。在上学期浅薄学习了数据库相关知识后，我打算进阶一下，不止局限于SQL语法的增删改查，更多地面向底层逻辑的知识拓展。本文简单地讲解了关于LSM-tree和B-tree的知识，也算我个人的一个学习总结。之后也许大概可能会有一篇实操性的文章吧，不确定。

WARNING!

由于笔者的文学功底和能力水平极其平庸，本文可能会不可避免地出现：

- 对错参杂地理解
- 外星人地语言表达
- 无意义的加粗加黑

请谅解

Refence

在学习过程，参考了以下书籍和网站

[《Designing Data-Intensive Applications》中文译版](#)

数据结构 C语言第三版 人民邮电出版社

[rocksDB文档](#)

[levelDB文档](#)

初级的驱动数据库的数据结构

世界上最简单的数据库

它非常简单，就是一个文本文件，然后每一行包含用户所输入的键值对，键值对之间会用逗号隔开。它只需要两串 `bash` 命令就可以实现。

```
#!/bin/bash

db_set () {
    echo "$1,$2" >> database
}

db_get () {
    grep "^$1," database | sed -e "s/^$1,/" | tail -n 1
}
```

每次添加或者是更改数据，都只是在文件的末尾新添一条记录，也就是说如果你是进行的更改操作，旧的值是不会被删除的。这样的模式类似于日志（log），一个仅追加的文件。

日志

引用《Designing Data-Intensive Applications》来解释**日志**（log）。“这个词通常指应用日志：即应用程序输出的描述正在发生的事情的文本。”

本文在更普遍的意义下使用日志这一词：一个仅追加的记录序列。它可能压根就不是给人类看的，它可以使用二进制格式，并仅能由其他程序读取。

虽然这样数据库在写入时十分高效，但是我们可以很容易发现这个数据库有显然的**不足之处**：

1. 每次查找都会从头查找，如果存储的数据量过大会造成查找时间过长。
2. 它的数据量也许大概可以“无限”增加，造成硬盘空间没了。

这时我们便引用一个数据结构来高效地查找数据库中特定键的值：**索引**（index）。

索引

index的大致思想是通过保存一些额外的数据来作为一个指引路标来帮助用户来找到想要的数据。

本质是一个从主数据衍生的**额外的数据**，它能够有效的**缩短查询所需要的时间**，但是它可能会导致**写入时的性能很难超过简单的追加写入文件**。这是一个值得程序员们进行权衡的地方。

键值数据的索引

接下来介绍一种常见的索引：**键值数据的索引**。

但是首先我们要插播一个知识点：**散列映射（hash map）** 和 **散列表（hash table）**

哈希表/散列表：一个有限连续的地址空间，用以存储按**散列函数/散列映射**计算得到相应散列地址的数据记录。———《数据结构》第三版

我的理解是它通过一个映射函数，也就是散列函数配合码值得到了一个存储地址。这个存储地址就是哈希表数组的下标比如存入 Key = "Apple", Value = "Red"。我假设通过某个散列函数得到了一个结果3。然后我就将"Red"这个value存储到数组的3号位置。

tip：有时会出现冲突情况，这里不做详细解释。

对于查找数据，既是重新进行一遍运算，直接定位，便可以做到不遍历全部数据。

键值存储与在大多数编程语言中可以找到的 **字典（dictionary）** 类型非常相似，通常字典都是用 散列映射（hash map）或 散列表（hash table）实现的。

既然可以用散列映射来表示内存中的数据结构，那我们也可以尝试用它来**索引硬盘上的数据**：假设我们的数据存储仅仅只是一个追加写入的文件，那最简单的策略就是：得到一个散列映射（散列函数），每一个键都能得到对应的存储地址——一般情况下能。然后将指向**值的偏移量**！！！存储进去，不是值。当我们想要更新或者新增时，也只是重复以上步骤。它仍然没有做到删除旧的数据，但是它极大地提升了大数据量时的查询速率。

以上方法看上去十分的简单，但是它是一个现实可行的方法。在现实中**Bitcask**（Riak中默认的存储引擎）就是这样做的。

Bitcask将键放在内存中，值放在硬盘中，值可以使用比内存更多的空间，而且只需要通过知道偏移量，进行一次硬盘查找来查询。像 Bitcask 这样的存储引擎非常适合每个键的值经常更新的情况。

简单的优化

但是我们目前所有的前提都是只进行追加写入，硬盘空间迟早会被用完。一种好的解决方法：将日志分为几个特定大小的段

当日志的长度达到我们所规定的长度时，停止写入当前的段。在另一个新的段中进行写入。

我们便可以对那些已经达到长度的段进行**压缩**，他会保留键值的最新更新，而且还会对多个段进行合并。那些旧的键值就被删去了。

每个段现在都有自己的内存散列表，将键映射到文件偏移量。为了找到一个键的值，我们首先检查最近的段的散列映射；如果键不存在，我们就检查第二个最近的段，依此类推。

合并过程将保持段的数量足够小，所以查找过程不需要检查太多的散列映射。

在整个想法实现过程中会有几个问题：

1. 文件格式。
2. 删除记录。
3. 崩溃恢复。
4. 部分写入记录
5. 并发控制。

追加和分段合并都是顺序写入操作，通常比随机写入快得多，尤其是在磁性机械硬盘上。

在面对范围查询是效率较低，而且散列表必须要能放在内存中，否则一切都是白谈。

SSTables和LSM-tree

SSTables

之前的追加写入，都是直接在尾部新增数据，对键值对的顺序并没有一个排序。现在我们要进行一个改变即键值对在文件中的序列要按照键排序。这个变化，便形成了一种新的格式：**排序字符串表（Sorted String Table）**。

和之前的没有排序的情况相比：

1. SSTable在合并时，更加地高效。它只需要**并排读取多个段，查看每个段的第一个键**，将排序最小的键安置到合并后的段。不断重复此步骤，这样也能保证合并后的段是按键排序的。这时就有人要问了：主包主包，如果在几个段中出现相同的键怎么办呢？这时不同段的"冻结时间"就起到了至关重要的作用，假设我们总是依次合并相邻的段，晚"冻结"的段的键总是比先"冻结"的段的键更"新"。所以，我们可以**保留晚"冻结"也就是更近的一个段的值，并且删除丢弃掉旧段中的值**。
2. SSTable**不用保存所有键的索引**。因为SSTable中总是按键来进行排序的。举个栗子：假如我们正
在内存中寻找键77，虽然我们不知道这个键在段文件中的确切偏移量。但是我们却知道比如70和80
的偏移。因为按键排序所以77必须出现在这两者之间。这代表着，我们可以从70的偏移位置查询，
直到找到77或者扫到80（这代表该段中没有这个键）也就意味着我们不再需要散列映射，只需要进
行稀疏索引。
3. 由于读取请求无论如何都需要扫描所请求范围内的多个键值对，因此可以将**这些记录分组为块（block）**，并在将其写入硬盘之前对其进行压缩
稀疏内存索引中的每个条目都指向压缩块的开始处。除了节省硬盘空间之外，压缩还可以减少对 I/O
带宽的使用。

构建和维护SSTables

这一段直接叙述《Designing Data-Intensive Applications》中的过程，因为它写的非常浅显易懂

我们可以让我们的存储引擎以如下方式工作：

1. 有新写入时，将其添加到内存中的平衡树数据结构（例如红黑树）来实现按键序排列。这个内存树有时被称为 内存表（memtable）。
2. 当内存表大于某个阈值（通常为几兆字节）时，将其作为 SSTable 文件写入硬盘。这可以高效地完成，因为树已经维护了按键排序的键值对。新的 SSTable 文件将成为数据库中最新的段。当该 SSTable 被写入硬盘时，新的写入可以在一个新的内存表实例上继续进行。

3. 收到读取请求时，首先尝试在内存表中找到对应的键，如果没有就在最近的硬盘段中寻找，如果还没有就在下一个较旧的段中继续寻找，以此类推。
4. 时不时地，在后台运行一个合并和压缩过程，以合并段文件并将已覆盖或已删除的值丢弃掉。这个方案效果很好。它只会遇到一个问题：如果数据库崩溃，则最近的写入（在内存表中，但尚未写入硬盘）将丢失。为了避免这个问题，我们可以在硬盘上保存一个单独的日志，每个写入都会立即被追加到这个日志上，就像在前面的章节中所描述的那样。这个日志没有按排序顺序但这并不重要，因为它的唯一目的是在崩溃后恢复内存表。每当内存表写出到 SSTable 时，相应的日志都可以被丢弃。

LSM-tree

基于前文的1) 只增不改。2) SSTable顺序排序。3) 后台合并。的原理的存储引擎就是所谓**LSM存储引擎**。而所谓**LSM-tree**是基于前文的日志结构文件系统来构建的。

这里要注意LSM存储引擎和LSM-tree的区别

1) LSM-tree仅是一个抽象的**数据结构概念**

2) LSM存储引擎是一个**具体的软件模块**是基于LSM-tree设计出来的具体“实物”

现代的levelDB和RocksDB都是非常纯正的LSM存储结构。

和前文一样，任何的存储引擎在实践中都面临着优化问题。比如面临一个极端问题：查找一个不存在的键，这时LSM-tree可能很慢。

这时为了优化，便出现一个概念**布隆过滤器（Bloom filters）**

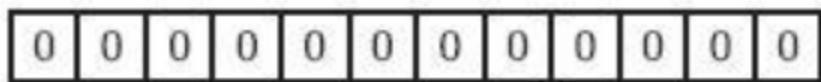
布隆过滤器《leveldb-handbook》：这里我们仅简单涉及布隆过滤器的概论和一小部分原理，关于它的具体数学结论，准确率不做详细追究，感兴趣的读者可以查看

<https://blog.csdn.net/jiaomeng/article/details/1495500> 此文。

布隆过滤器是一种空间效率极高的随机数据结构。利用位数组很简洁地表示一个集合，并能判断一个元素是否属于这个集合

但是它偶尔会出错，可能把不属于这个集合的元素误认为属于这个集合。

具体原理：bloom过滤器底层是一个位数组，初始时每一位都是0



当插入值x后，分别利用k个哈希函数（图中为3）利用x的值进行散列，并将散列得到的值与bloom过滤器的容量进行取余，将取余结果所代表的那一位值置为1。



一次查找过程与一次插入过程类似，同样利用k个哈希函数对所需要查找的值进行散列，只有散列得到的每一个位的值均为1，才表示该值“有可能”真正存在；反之若有任意一位的值为0，则表示该值一定不存在。例如y1一定不存在；而y2可能存在。



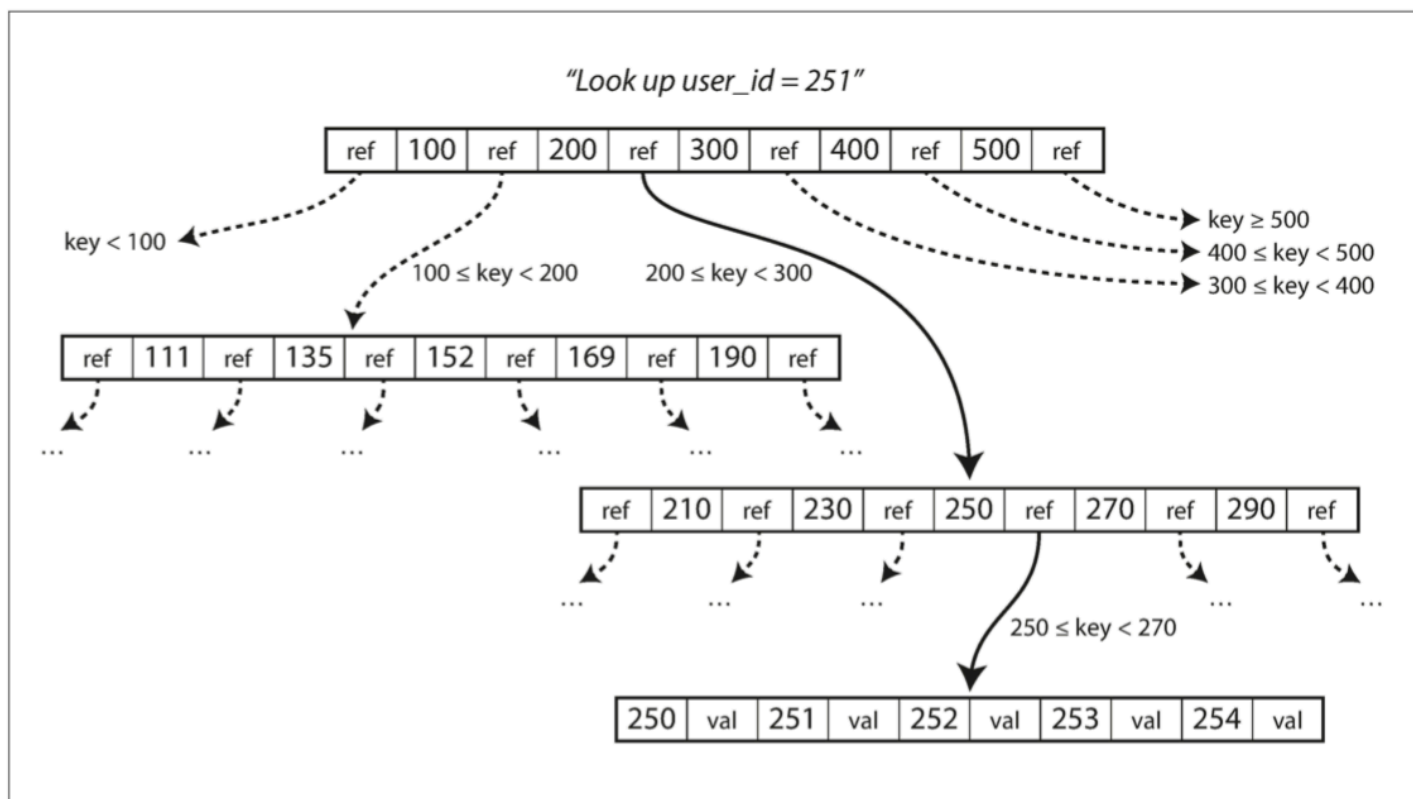
B-tree

前文所讲的索引类型目前并不算是最常见的索引类型，目前最广泛的索引结构与其相当不同，我们叫它 **B-tree**。像 SSTables 一样，B 树保持按键排序的键值对，这允许高效的键值查找和范围查询，但这也是所有的相似之处了。

前文的日志结构索引将数据库分解为等大的段，通常会有几mb，而且总是顺序写入段，但是，而且总是顺序写入段，但是 B-tree则是将数据库分解成固定大小的块通常只有4kb，并且一次只能读取或写入一个页面。

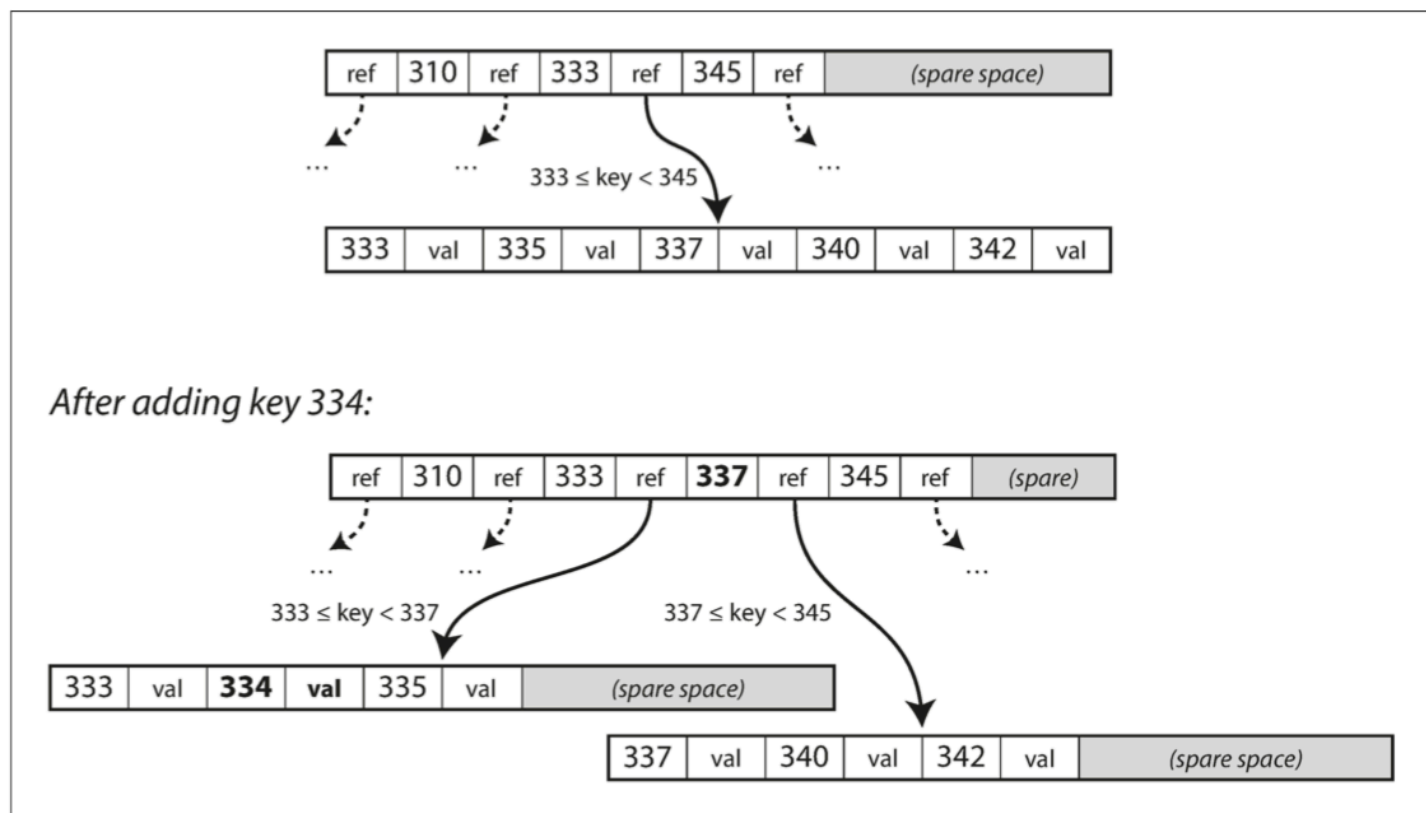
每个页面都可以使用地址或位置来标识，这允许一个页面引用另一个页面 —— 类似于指针，但在硬盘而不是在内存中。

下图讲解了如何利用B-tree来快速查找一个键：



这里有一个注意点：当我们想添加一个键时，我们找到了其范围能包含这个键的页面，但如果这个页面没有空间来容纳了，这是就需要B-tree的生长，将其继续分为两个半满页面。

如下图所示：



B-tree的写操作和LSM-tree**截然不同**，它是用**新数据来覆盖掉旧的数据**，而LSM-tree仅是**追加写入**，然后最后删除旧数据，并不修改已有的内容。

在机械硬盘上，我们要等待磁头移动到正确的位置，磁盘转过去才会进行覆写。而在固态硬盘上由于SSD 必须一次擦除和重写相当大的存储芯片块，所以会发生更复杂的事情

问题优化

有时可能会出现一种情况，在进行上文提到的页面分裂时，整个分裂过程没有完成就发生比如断电之类的事情，这时可能会出现父页面仍指向我们需要分裂的那个页面，但是我们的页面早已分裂，新分裂的两个页面，没有被指向，从而出现问题。这时一个额外的数据结构便挺身而出：**预写式日志-WAL**这是一个仅追加的文件，每个 B 树的修改在其能被应用到树本身的页面之前都必须先写入到该文件。当数据库在崩溃后恢复时，这个日志将被用来使 B 树恢复到一致的状态。

如果多个线程要同时访问 B 树，则需要仔细的并发控制—— 否则线程可能会看到树处于不一致的状态。这通常是通过使用 **锁存器（latches，轻量级锁）保护树的数据结构来完成。**

这里简单讲一下锁存器：在某个动作完成之前，把相关的路径全部“闷住”，保证别人看到的树永远是结构完整的。直到完成了，再进行修改。

B-tree现在也存在许多优化版本，这里不做详细介绍，读者可以下去自行查询。如B+树。

LSM-tree和B-tree的比较

在这之前我们还要介绍一下读放大，写放大，空间放大

读放大：为了读取一条数据，数据库实际从磁盘读取的数据页数（或字节数）。举个栗子：我想在数据库里查询一个姓名，为了找到它，从磁盘读了8个不同的数据块，读放大就是8。

写放大：实际写入磁盘的数据量与应用程序请求写入的数据量之比。举个栗子：你想存 1KB 的数据，但数据库最终往硬盘写了 10KB，写放大就是 10

空间放大：数据库占用的磁盘空间与实际数据量之比。举个栗子：例子：你的实际数据只有 1GB，但数据库文件占了 2GB 磁盘空间，空间放大就是 2。

B-tree：优先优化了 R（读），牺牲了 U（写）和 M（空间）。

LSM-tree：优先优化了 U（写）和 M（空间），牺牲了 R（读）。

按照经验来说，**LSM-tree的写入速度优于B-tree，但是B-tree的读取速度又明显优于LSM-tree。**

LSM-tree通常能够比 B-tree支持更高的写入吞吐量，部分原因是它们有时具有较低的写放大；部分是因为它们顺序地写入紧凑的 SSTable 文件而不是必须覆写树中的几个页面。

LSM-tree能比B-tree被压缩的更好，因为B-tree存储引擎会由于一些页面的空间未被使用而造成所谓的“浪费”；但是LSM-tree不会，而且它还会定期删去旧数据进行合并等。

不过LSM-tree也存在一些问题：压缩过程可能影响正在进行的读写操作。就是因为硬盘的资源有限，有时可能出现因为写入量过大导致前面的段还没有压缩，合并完，导致未合并的段越来越多从而使得磁盘空间用完等问题。

B-tree的一个优点是每个键只存在于索引中的一个位置，使得其在想要提供强大的事务语义的数据库中很有吸引力。

尾记

只涉及了一些皮毛，接下来会继续深入，包括实操啥的，这篇文章算是纯概念了。